

Design Proposal — Legacy .NET Application Migration to AWS

Assumptions and MVP Scope

This design assumes the first goal is a low-risk migration of a legacy .NET application with minimal application changes. For the MVP, I would prioritise ECS Fargate, ALB, private subnets, RDS SQL Server Multi-AZ, Secrets Manager, CloudWatch logs, and a basic GitHub Actions → ECR → ECS deployment path. Services such as WAF, Security Hub, Config, read replicas, and Aurora migration are production-hardening or future optimisation items rather than required for the first deployment.

1. AWS Architecture Overview

(See architecture diagram below)

2. Core Infrastructure

Compute — Amazon ECS on Fargate

Decision	Rationale
ECS Fargate over EKS	Lower operational overhead for an MVP; no cluster nodes to manage. Fargate is fully serverless, so the team can focus on the application rather than Kubernetes.
Application Load Balancer (ALB)	Layer-7 routing, health checks, and TLS termination. Enables blue/green or rolling deployments natively with ECS.

Container Base Image

Image	Platform	When to use
<code>mcr.microsoft.com/dotnet/aspnet:8.0</code>	Linux	Recommended — for apps ported to modern .NET 8+. Cheapest Fargate pricing, fastest startup, broadest region support.
<code>mcr.microsoft.com/dotnet/aspnet:8.0-alpine</code>	Linux	Even smaller (~100 MB), suitable when there are no native C-library dependencies.
<code>mcr.microsoft.com/dotnet/framework/aspnet:4.8</code>	Windows	Only if the app cannot be ported off legacy .NET Framework — requires Windows Fargate (more expensive, fewer regions).

For the MVP we recommend targeting **.NET 8 on Linux** (`mcr.microsoft.com/dotnet/aspnet:8.0`) to benefit from lower costs and broader AWS feature support. If the app must remain on .NET Framework 4.x short-term, use the Windows image as a temporary measure with a clear migration path to .NET 8.

Networking — Amazon VPC

- **Two Availability Zones** for high availability.
- **Public subnets** host the ALB.
- **Private subnets** host ECS tasks and the database — no direct internet access.
- **NAT Gateway** (one per AZ for production; one shared for MVP) allows outbound internet access for pulling images and patches.
- **VPC Flow Logs** sent to CloudWatch for auditability.

Data — Amazon RDS for SQL Server (or Aurora)

Option	When to choose
Amazon RDS for SQL Server	Direct compatibility with existing T-SQL, stored procedures, and SSIS packages — minimal code changes.
Amazon Aurora PostgreSQL (long-term)	If the team is willing to refactor queries; better cost-performance ratio and serverless scaling.

For the MVP we recommend **RDS for SQL Server Multi-AZ** to minimise migration risk.

3. Data Migration Strategy

On-Prem SQL Server → AWS DMS (full load + CDC) → RDS SQL Server (Multi-AZ)

1. **Assessment** — Run AWS Schema Conversion Tool (SCT) to identify incompatibilities.
2. **Replication instance** — Provision an AWS Database Migration Service (DMS) replication instance in the target VPC.
3. **Full load** — Perform an initial bulk migration of all tables.
4. **Change Data Capture (CDC)** — Keep the source and target in sync during the cut-over window.
5. **Validation** — Use DMS data validation and run application integration tests against the new database.
6. **Cut-over** — Update the application's connection string, switch DNS, and decommission the DMS instance.

4. Security

Layer	Service / Practice
Network	Security Groups (least-privilege), NACLs, private subnets for compute & data
Edge protection	AWS WAF on ALB to filter OWASP Top 10 attacks, AWS Shield Standard for DDoS
Secrets	AWS Secrets Manager for DB credentials; rotated automatically
Identity	IAM roles for ECS task execution with least-privilege policies; no long-lived credentials
Encryption	TLS 1.2+ on ALB; EBS/RDS encryption at rest with AWS KMS
Container security	ECR image scanning on push; non-root user in Dockerfile
Monitoring	Amazon GuardDuty for threat detection; AWS CloudTrail for API audit logs
Compliance	AWS Config rules to continuously evaluate resource configuration

5. CI/CD Pipeline

Developer push → GitHub Actions → Build & Test → Push to ECR → CDK Deploy → EC

Pipeline stages:

1. **Source** — Triggered on push to `main` branch in GitHub.
2. **Build** — Dockerfile is built and tagged with the Git SHA.
3. **Test** — Unit tests and container structure tests run inside the pipeline.
4. **Push** — Image is pushed to Amazon ECR (scanned automatically).
5. **Deploy Infra** — `cdk deploy` provisions / updates all CloudFormation stacks.
6. **Deploy Service** — ECS service is updated to the new task definition; ALB health checks gate the rollout.
7. **Post-deploy** — Smoke test against the ALB endpoint; CloudWatch alarms auto-rollback if failures exceed threshold.

Environments

For production readiness the pipeline would be extended to:

- **dev** → automatic deploy on every push
- **staging** → deploy on merge to `main`, manual approval gate
- **production** → promoted from staging after approval

6. Scalability Considerations

- **ECS Service Auto Scaling** — Target-tracking on CPU/memory metrics to handle load spikes.
 - **ALB scaling** — Managed automatically by AWS.
 - **RDS Read Replicas** — Add read replicas for read-heavy workloads.
 - **Aurora Serverless** — If migrating to Aurora, scale to zero during idle periods.
 - **Centralised logging** — CloudWatch Logs with Log Insights; optionally ship to OpenSearch for full-text search.
 - **Centralised security monitoring** — AWS Security Hub aggregates findings from GuardDuty, Inspector, Config, and IAM Access Analyzer into a single pane of glass.
-

7. Trade-offs

- **ECS Fargate over EKS** — The scenario asks for an MVP and the team likely benefits from lower operational overhead. EKS would be warranted if the organisation already has Kubernetes expertise or needs multi-cloud portability, but it adds complexity that is not justified for a single-application migration.
- **RDS SQL Server first, Aurora later** — RDS SQL Server minimises database migration risk for a legacy .NET workload because existing T-SQL, stored procedures, and SSIS packages work without modification. Aurora PostgreSQL could be considered later, but only after assessing stored procedures, T-SQL dependencies, and refactoring effort.
- **Private subnets + NAT Gateway over public subnets** — Adds additional monthly cost per NAT Gateway plus data processing charges, but prevents direct inbound internet access to containers (defense in depth). For a strict MVP budget, placing ECS tasks in public subnets with restrictive security groups is an acceptable alternative.
- **Terraform over CDK or CloudFormation** — Terraform was chosen for the design proposal because it provides a declarative, cloud-agnostic approach to infrastructure-as-code that is platform-independent and highly portable. The HCL syntax is intuitive and widely adopted across teams. CDK is a strong alternative if the team prefers defining infrastructure in a programming language (C#/.NET or TypeScript).

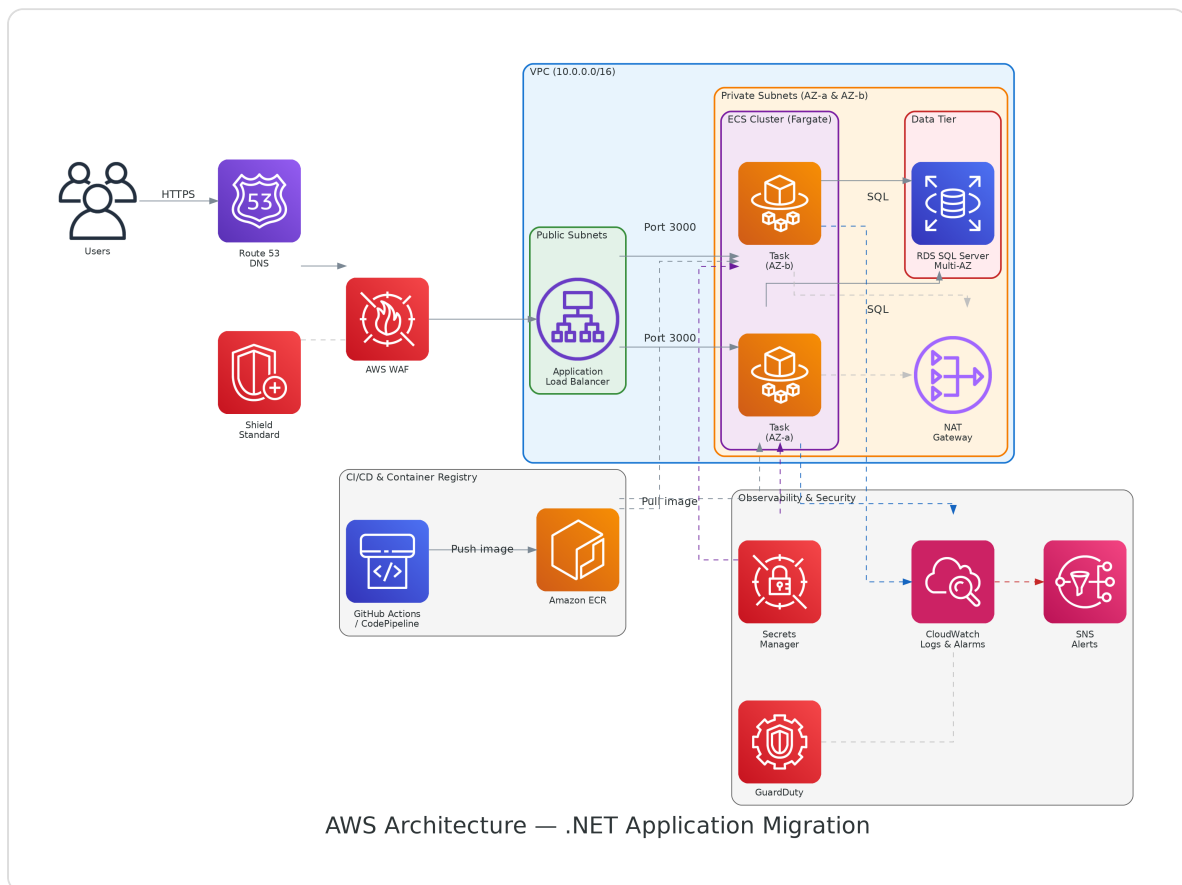


Figure 1 — Target AWS architecture for the containerised .NET application